Formative activity 1

In the first exercise, the code creates a perceptron, which receives two inputs (45 and 25) with two weights (0.7, 0.1) and produces a binary output (0 or 1).

The weights define the importance of the inputs, meaning that, in this case, the first input has a higher weight than the second one.

Then the code calculates the dot product, and with the step function, the perceptron makes a decision: if the sum is >1, the output will be 1; if the sum is <1, the output will be 0.

In the last lines, the weights are changed to -0.7 and 0.1, and consequently, the output changes.

To continue the exercise, I decided to change the inputs and the weights.

First, I swapped the input that became inputs = np.array([20, 45]), maintaining the weights. The result shows a change in the values of the weighted sum from 33.5 to 18.5.

Then, I changed the weights respectively from 0.7 and 0.1 to 0.3 and 0.8, and I changed the inputs to the original values. In this case, what changes is the relative importance: the second input will have more importance than the first one in determining the final result. In both cases, the output is the same (1), but the way in which the perceptron arrives at it is different.

Formative Activity 2

In the second exercise, the perceptron is trained to simulate a logical operator AND.

I decided not to modify the inputs because it is a matrix, but I changed the weights.

I modified the initial weights from 0.0 to 0.5. This will change the perceptron's starting point and how the weights are updated during training.

The results show that the total error is 0, indicating that there is no need to update the weights during training. This means that the chosen weights already classify all the AND input combinations correctly. In addition, the change did not affect the final result, but it eliminated the need for further learning.

The second change will be the learning rate. The weight will again be 0.0 for both variables, while the learning rate will be 0.6. During the training, the perceptron has updated the weights from 0.0 to 0.6, and it has achieved a total error equal to 0 quickly. This shows that a higher learning rate allows larger weight updates, and this can accelerate the learning process. The model continues to classify the AND logical function correctly.

In conclusion, in both cases, the result did not change. However, by modifying the initial weights, the model was able to classify the input correctly without updates. While increasing the learning rate, the model needed to learn, but it had to correct the weights quickly thanks to larger weight updates.

Formative Activity 3

The third activity is a Multilayer Perceptron (MLP), which is a neural network with 2 input neurons (x1 and x2), a hidden layer with 3 neurons, and 1 output neuron.

In the exercise, the weights are initialised randomly, so I decided to change only the learning rate and the epochs.

To investigate the effect of the learning rate, I reduced it from 0.6 to 0.3 while keeping the number of epochs constant at 400,000. The training error decreased more slowly,

indicating that a learning rate of 0.6 provided faster convergence while maintaining stable training.

Finally, to investigate the effect of the number of epochs, I reduced the value from 400,000 to 200,000 while keeping the learning rate constant at 0.6. The network continued to learn, as the training error decreased during the training. Decreasing the number of epochs gives the model more opportunities to update its weights and improve its performance, but the rate of improvement may decrease.

In conclusion, this exercise showed that both the learning rate and the number of training epochs influence the learning process of a multilayer perceptron. It is important to choose the appropriate values for the hyperparameters to obtain stable model training.